Protocol-Layer Governance for LLM Agents

Identity-Bound Oversight with Zero-Modification Deployment and Multi-Hop Authority Propagation

Marcelo Fernandez
TraslaIA
info@traslaia.com

Abstract

Governing an LLM agent at runtime requires identity-bound, cryptographically verifiable human authorisation to resolve persistent halt states (P8). The deployment question that remained open is: *how does the governance stack reach an agent without modifying the agent's code?*

We answer this question by positioning the governance stack at the *Model Context Protocol* (MCP) layer. We introduce the *MCP Governance Proxy* (Π), a stateful *protocol-layer governance control plane* that intercepts tool calls between any MCP-compatible agent client and any MCP server, applying the P7–P8 governance stack (IML + RAM + Recovery Loop + APB) transparently. No modification to the agent client or server is required; the proxy is a drop-in addition to the deployment topology. This positions governance at the protocol boundary rather than inside the agent—the same architectural move that service meshes make for microservice traffic and that zero-trust gateways make for API calls.

We prove three theorems. *Transparency Invariance* (T9.1) establishes that on the non-halt path the proxy is observationally equivalent to a direct connection: the agent's tool-call results are unchanged. *Halt Latency Bound* (T9.2) establishes that a governance event at the proxy propagates a halt signal to the agent within a latency bounded by $\Delta_{\text{net}} + \Delta_{\text{verify}} + \Delta_{\text{policy}}$, all of which are small relative to LLM inference times. *Multi-Hop Authority Propagation* (T9.3) establishes that in Agent-to-Agent (A2A) delegation chains, when a sub-agent's tool call triggers a governance event, the required APB has well-defined binding semantics: the accountable principal is the one registered for the *originating* agent, and the delegation chain is recorded in the evidence block.

We validate the construction through five experiments. *E1* (latency overhead): 10,000 governed tool calls; the O(1) windowed IML monitor achieves a P95 overhead of 51.8 µs, well below the 10 ms T9.1 gate. *E2* (real-agent APB, 5 seeds × 200 steps): 310 HALT events with 100% APB validity; T9.1 holds. *E3* (multi-hop A2A, depths 1–5): 334 HALT events across all depths, 100% originator binding; T9.3 holds for all chain lengths. *E4* (concurrency): $N \in \{1, 4, 16, 64\}$ threads, 0 exceptions, 100% APB validity; true-parallel process mode validated up to $N = 16$. *E5* (security adversarial suite): five attack vectors (wrong-key signing, evidence tampering, replay, revoked-principal, authority substitution); adversary success rate = 0%.

1 Introduction

The *Model Context Protocol* (MCP) [1] has emerged as a widely adopted protocol for connecting LLM-based agents to external tools and data sources. An agent that speaks MCP can call any MCP-compatible server without custom integration; the protocol handles discovery, invocation,

and result delivery uniformly. This uniformity creates a deployment opportunity that prior work in this series did not exploit: because all tool calls pass through a single, well-specified protocol, a governance layer inserted at that protocol boundary governs *all* of the agent's external actions without modifying the agent's code—instantiating, in effect, a *protocol-layer governance control plane* for LLM agents.

Paper 7 of this series [2] established an integrated runtime governance stack that detects drift, applies the Reconstructive Authority Model (RAM), and escalates persistent halt events for human resolution. Paper 8 [3] formalised the *Accountability Proof Block* (APB) as the identity-bound, cryptographically signed record of those human resolutions. Both papers explicitly identified MCP and A2A integration as future work (P7 §13, P8 §10). This paper fulfills that commitment.

Problem. Deploying the P8 governance stack on an existing LLM agent requires either (a) modifying the agent's source code to call the governance layer at each tool invocation, or (b) wrapping the agent at a protocol boundary that the agent already respects. Option (a) is impractical for agents whose code is unavailable, large, or managed by a third party. Option (b) is exactly what MCP enables.

Contributions.

1. We define the *MCP Governance Proxy* Π formally (§4.2), giving its state, its per-call decision function, and its interaction with the P8 APB protocol extension.
2. We prove three theorems (§5): Transparency Invariance (T9.1), Halt Latency Bound (T9.2), and Multi-Hop Authority Propagation (T9.3).
3. We provide a Python implementation on the official `mcp` SDK (§6), supporting both stdio and HTTP+SSE transports, with 92 tests (61 inherited from P7+P8, 31 new).
4. We validate the construction empirically through five experiments (§7–§11): E1 latency overhead, E2 real-agent APB end-to-end, E3 multi-hop A2A authority propagation (depths 1–5), E4 concurrency stress (threads and processes), and E5 security adversarial suite (five attack vectors rejected).

Independence. The formal framework, theorems, and experiments in this paper are self-contained; this paper can be read standalone assuming only §3. P8's APB is used as an imported dependency (frozen baseline), not re-proved. Readers unfamiliar with P7 or P8 will find the necessary definitions restated there.

Roadmap. §2 surveys related work. §3 restates the minimal P7+P8 prerequisites. §4 defines the proxy formally. §5 proves T9.1–T9.3. §6 describes the implementation. §7–§11 report the five experiments. §12 discusses security analysis, limitations, and extensions. §13 concludes.

2 Related Work

MCP and agent middleware. The Model Context Protocol [1] defines a client–server protocol for tool discovery and invocation over stdio and HTTP+SSE transports. Existing MCP

deployments do not include a governance layer; each tool is exposed and invoked directly. The Agent-to-Agent (A2A) protocol [4] extends this to multi-agent settings, allowing one agent to delegate tasks to another. Neither protocol specifies how governance or accountability are maintained across delegation chains.

Agent orchestration frameworks. LangGraph [5], AutoGen [6], CrewAI [7], and OpenAI Swarm [8] provide orchestration primitives for multi-agent workflows. They do not provide identity-bound, cryptographically verifiable records of human authority decisions at runtime. Our proxy is orthogonal: it can be inserted into any of these frameworks at the MCP layer without modifying the orchestration logic.

Runtime verification. Runtime monitoring [9–11] checks that running systems satisfy formal specifications. Existing monitors observe system behaviour but do not provide a mechanism for human authority decisions to be cryptographically bound to the events that triggered them. The proxy extends runtime monitoring with the identity-binding of the APB.

Middleware and proxies. The Proxy design pattern [12] is a standard approach for interposing behaviour between a client and a server. Existing governance proxies for web services (API gateways, service meshes) enforce access control policies but do not maintain per-event, per-principal accountability records. Our contribution is specific to the LLM-agent governance problem: stateful drift detection, the RAM authority model, and the APB all require state that general-purpose proxies do not maintain. The present work occupies the same architectural category—a *governance control plane at the protocol layer*—but for LLM-agent systems, with state requirements specific to that domain.

Agent governance series. This paper is the ninth in a series [2, 3, 13–18]. The series develops a formal governance framework from first principles; this paper addresses the *deployment* question that completes the framework.

3 Background

We restate the minimal prerequisites from P7 and P8. Readers familiar with both papers may skip to §4.

3.1 Design Constraints

Two design constraints from P7 [2] are inherited verbatim:

DC.1 (Separation of Recalibration Authority). For all t, $\mathcal{A}_0(t) = \mathcal{A}_0(0)$. The execution layer is forbidden from modifying its own admission baseline at runtime.

DC.2 (Drift Event Classification). A halt is *transient* if the drift estimator $\widehat{D}(\tau) \geq \theta$ for τ within a bounded window ΔT and returns below θ thereafter; it is *persistent* if $\widehat{D}(t) \geq \theta$ for all $t \in [t_0, t_0 + \Delta T]$. Persistent halts constitute *governance events*.

3.2 Accountability Proof Block (APB)

From P8 [3], an APB signed by principal H_i is:

$$\text{APB} \;=\; (E_s,\; D_h,\; \sigma_h)$$

where $E_s = (\text{hash}(\mathcal{A}_0),\, \widehat{D}(t_e),\, t_e,\, \text{hash}(\text{trace}_{\leq t_e}),\, \text{cause})$ is the system evidence block, $D_h = (H_i,\, \text{decision},\, \text{rationale},\, \text{scope})$ is the human decision block, and $\sigma_h = \text{Sign}_{sk_i}(\text{canon}(E_s) \| \text{canon}(D_h))$ is the ed25519 signature.

A verifier checks five predicates V1–V5: (V1) signature validity, (V2) principal registered, (V3) principal active at t_e, (V4) t_e within replay window, (V5) event_id not previously seen (replay resistance, introduced in P9). Theorems T8.1–T8.3 of P8 establish governance completeness, non-repudiability, and impossibility of anonymous re-authorisation.

3.3 Governance Stack

The governance stack processes each tool call through three layers:

1. **IML Monitor**: computes $\widehat{D}(t)$ from the running tool-call trace using JS-divergence (D_t), mean risk score (D_c), and depth deviation (D_l), combined with EMA smoothing.
2. **RAM Gate**: for tool calls with risk score RS $\geq$ 45, checks authority components $\{I, B, R, C, E\}$ under partial observability and returns `ADMIT`, `HALT`, or `DENY`.
3. **Recovery Loop**: on `HALT`, attempts up to k iterations with increasing state coverage. Returns `RESUME`, `HALT`, or `ESCALATE`.

Persistent `HALT` or `ESCALATE` from the Recovery Loop constitutes a governance event requiring an APB.

4 Formal Framework

4.1 MCP Tool Call Semantics

Let Σ be a finite alphabet of tool names and $\mathcal{A}$ the space of argument dictionaries. A tool server $\mathcal{S}$ is a function $\mathcal{S} : \Sigma \times \mathcal{A} \to \mathcal{R}$ mapping calls to results. A *direct connection* between client and server produces:

$$\text{direct}(c) = \mathcal{S}(c.\text{name},\, c.\text{args}) \quad \text{for a call } c = (\text{name}, \text{args}).$$

Definition 4.1 (Tool Call Trace)**.** *A* tool call trace $\tau = (c_1, r_1, \ldots, c_n, r_n)$ *is an alternating sequence of calls* $c_i \in \Sigma \times \mathcal{A}$ *and results* $r_i \in \mathcal{R}$*, produced in session order. The* result trace *of* τ *is* $\rho(\tau) = (r_1, \ldots, r_n)$*.*

4.2 MCP Governance Proxy

Definition 4.2 (Proxy Session State)**.** *A* proxy session state $s \in \mathcal{S}_\Pi$ *is a tuple*

$$s = (\tau,\; \mathcal{A}_0,\; \widehat{D}_{ema},\; \mathcal{E}_{pend})$$

where τ is the current trace (4.1), $\mathcal{A}_0$ is the admission snapshot (fixed at session initialisation), $\widehat{D}_{ema}$ is the EMA-smoothed drift estimate, and $\mathcal{E}_{pend}$ is the map of pending evidence blocks awaiting APB responses (evidence id $\to E_s$). The initial state $s_0 = (\varepsilon,\ \mathcal{A}_0^{(0)},\ 0,\ \emptyset)$.

Definition 4.3 (Governance Decision Function)**.** *The* governance decision function govern : $\mathcal{S}_\Pi \times \Sigma \times \mathcal{A} \to \{\texttt{ADMIT},\ \texttt{HALT},\ \texttt{DENY}\}$ *is:*

$$
\begin{aligned}
s' &= \text{update}(s,\ name,\ args) \quad \textit{(append to } s.\tau\textit{, recompute } \widehat{D}\textit{)} \\
d_{RAM} &= \text{RAMGate}(name,\ RS(name),\ s'.\widehat{D}_{ema}) \\
d &= \begin{cases} \texttt{ADMIT} & \textit{if } d_{RAM} = \texttt{ADMIT} \\ \texttt{DENY} & \textit{if } d_{RAM} = \texttt{DENY} \\ \text{RecoveryLoop}(d_{RAM},\ s'.\widehat{D}_{ema},\ \ldots) & \textit{if } d_{RAM} = \texttt{HALT} \end{cases}
\end{aligned}
$$

where $\text{RS} : \Sigma \to [0, 100]$ *is the risk score mapping.*

Definition 4.4 (MCP Governance Proxy)**.** *An* MCP Governance Proxy Π *for principal set $\mathcal{P}$ is a stateful function:*

$$\Pi : \mathcal{S}_\Pi \times \Sigma \times \mathcal{A} \ \to\ \mathcal{R} \ \cup\ \{p9/\text{apbRequired}(E_s,\ eid)\} \ \cup\ \{\text{GOVERNANCE\_DENY}\}$$

On receiving call $c = (name, args)$ in state s:

1. *Compute* $d = \text{govern}(s,\ name,\ args)$.
2. *If* $d = \texttt{ADMIT}$*: forward c to server, return $r = \mathcal{S}(name, args)$, update s.*
3. *If* $d \in \{\texttt{HALT}, \texttt{ESCALATE}\}$*: construct* $E_s = \text{construct\_evidence}(s',\ cause)$*, assign* $eid \leftarrow \text{fresh\_id}()$*, record* $s'.\mathcal{E}_{pend}[eid] \leftarrow E_s$*, emit notification* $p9/\text{apbRequired}(E_s,\ eid)$ *to client.*
4. *If* $d = \texttt{DENY}$*: return* GOVERNANCE_DENY.

On receiving $p9/\text{apbResponse}(eid,\ \text{APB})$*:*

1. *Retrieve* $E_{s,\text{exp}} = s.\mathcal{E}_{pend}[eid]$.
2. *Verify* $\text{APB}.E_s = E_{s,\text{exp}}$ *and* $\text{verify}(\text{APB}, \mathcal{P})$.
3. *If valid: remove from pending, execute* $\text{APB}.D_h.decision$.
4. *If invalid: emit* $p9/\text{apbRejected}(eid,\ reason)$.

Protocol extension. The three custom message types $p9/\text{apbRequired}$, $p9/\text{apbResponse}$, and $p9/\text{apbRejected}$ are JSON-RPC notifications transmitted over the existing MCP transport. Standard MCP clients that do not implement the extension ignore them (Remark 4.1).

Remark 4.1 (Safe Failure Under Legacy Clients)**.** *A standard MCP client that does not implement the P9 extension will treat* $p9/\text{apbRequired}$ *as an unknown notification and ignore it per the JSON-RPC specification [19]. The pending tool call will not receive a response, causing the client's call to time out. This is the appropriate behaviour: a non-P9-aware agent is halted, not silently resumed. The failure mode is therefore* safe *(no unauthorised execution) rather than silent.*

4.3 Multi-Hop Authority Delegation (A2A)

In Agent-to-Agent delegation, an originating agent A_1 delegates a task to a sub-agent A_2 (possibly recursively). The sub-agent makes tool calls to a server; the proxy governs those tool calls.

Definition 4.5 (Delegation Chain)**.** *A* delegation chain *of depth n is a sequence* chain $= (A_1, A_2, \ldots, A_n)$ *where A_1 is the* originating agent*, each $A_i \xrightarrow{\delta} A_{i+1}$ is a delegation event recorded in the IML trace, and A_n makes the proxied tool call. The originator* $\text{orig}(\text{chain}) = A_1$.

Definition 4.6 (Multi-Hop APB Semantics)**.** *For a governance event triggered by A_n's tool call in chain* $\text{chain} = (A_1, \ldots, A_n)$*, the required APB has:*

$$E_s.\mathit{cause} \supseteq \{\texttt{delegation\_chain} = \text{id}(\text{chain}),\ \texttt{originator} = \text{id}(A_1)\}$$
$$D_h.H_i \in \mathcal{P}_{\text{orig}(\text{chain})}$$

where $\mathcal{P}_{\text{orig}(\text{chain})}$ is the principal set registered for the originating agent A_1.

The semantics formalise the accountability principle: the human accountable for the originating agent's actions is the one who must authorise a recovery from any governance event caused by agents operating under that agent's authority.

5 Theorems

5.1 T9.1 — Transparency Invariance

Theorem 5.1 (Transparency Invariance)**.** *Let $\tau = (c_1, \ldots, c_n)$ be a sequence of tool calls processed by proxy Π in session state s_0, such that* $\text{govern}(s_i, c_{i+1}) = \texttt{ADMIT}$ *for all $i \in \{0, \ldots, n-1\}$. Then the result trace returned to the client is equal to the result trace produced by a direct connection to the server:*

$$\rho(\tau_\Pi) = \rho(\tau_{\mathit{direct}}).$$

Proof. By induction on n.

Base case ($n = 1$). Call $c_1 = (\text{name}, \text{args})$ with $d_1 = \texttt{ADMIT}$. By Definition 4.4 step 2, Π forwards c_1 to the server and returns $r_1 = \mathcal{S}(\text{name}, \text{args})$ unchanged. By definition of direct connection, $\text{direct}(c_1) = \mathcal{S}(\text{name}, \text{args}) = r_1$. Therefore $\rho(\tau_\Pi) = (r_1) = \rho(\tau_{\text{direct}})$.

Inductive step. Assume the claim holds for all sequences of length $n-1$. Consider c_n with $d_n = \texttt{ADMIT}$. By the inductive hypothesis, $(r_1, \ldots, r_{n-1})$ is identical under Π and direct connection. For call c_n: by Definition 4.4 step 2, Π forwards c_n to the server and returns $r_n = \mathcal{S}(c_n.\text{name}, c_n.\text{args})$ unchanged, regardless of the internal proxy state s_n (the state affects only the governance decision, which is `ADMIT` by hypothesis, not the forwarded call or response). By Lemma 5.2, the result is identical to $\text{direct}(c_n)$. Therefore $\rho(\tau_\Pi) = (r_1, \ldots, r_n) = \rho(\tau_{\text{direct}})$. □

Lemma 5.2 (MCP Response Identity Under Passthrough)**.** *For any tool call $c = (\mathit{name}, \mathit{args})$ and proxy state s, if* $\text{govern}(s, c) = \texttt{ADMIT}$*, then* $\Pi(s, c).\mathit{result} = \mathcal{S}(\mathit{name}, \mathit{args})$.

Proof. By Definition 4.4 step 2: on $d = \texttt{ADMIT}$, the proxy forwards the unmodified call (name, args) to the server and returns the server's unmodified response. No transformation is applied to either the call arguments or the result. □

Remark 5.1 (Non-halt path observational equivalence)**.** *Theorem T9.1 makes precise the claim that governance is* transparent *to non-P9-aware clients. The proxy introduces latency on the non-halt path (bounded in T9.2) but produces no semantic difference in the tool-call results.*

5.2 T9.2 — Halt Latency Bound

Theorem 5.3 (Halt Latency Bound)**.** *Let c be a tool call that triggers a governance event at the proxy at time t_0. The agent receives the halt signal (via p9/*apbRequired *or an error response) no later than*

$$t_0 + \Delta_{\text{policy}} + \Delta_{\text{net}},$$

and APB validation upon the client's response completes within an additional Δ_{verify}, so the total governance resolution latency is bounded by

$$\Delta_{halt} \;\leq\; \Delta_{\text{net}} + \Delta_{\text{verify}} + \Delta_{\text{policy}}.$$

Proof. By construction of Definition 4.4 and the governance stack.

Governance evaluation. On receiving c at t_0, the proxy runs govern(s, c): (i) The IML trace update is $O(|\tau|)$, bounded by the session trace length; we write this cost as $\Delta^{(1)}_{\text{policy}}$. (ii) The RAM Gate executes a constant number of pseudo-random draws for each of $|\{I, B, R, C, E\}| = 5$ components; time is $O(1)$, included in Δ_{policy}. (iii) The Recovery Loop executes at most $k_{\max}$ iterations (a compile-time constant), each $O(1)$; total $O(k_{\max}) \subseteq \Delta_{\text{policy}}$. The governance evaluation completes at $t_0 + \Delta_{\text{policy}}$.

Signal delivery. The proxy emits *p9/*apbRequired immediately after the governance evaluation (Definition 4.4 step 3). The message travels from proxy to client in network time Δ_{net}. The client receives the halt signal at $t_0 + \Delta_{\text{policy}} + \Delta_{\text{net}}$.

APB validation. When the client responds with *p9/*apbResponse, the proxy verifies the APB. By Lemma 5.4, verification is $O(1)$, denoted Δ_{verify}.

Total. Summing: $\Delta_{\text{halt}} = \Delta_{\text{policy}} + \Delta_{\text{net}} + \Delta_{\text{verify}}$. Rewriting in the bound's conventional order: $\Delta_{\text{halt}} \leq \Delta_{\text{net}} + \Delta_{\text{verify}} + \Delta_{\text{policy}}$. □

Lemma 5.4 (Verification Time is $O(1)$)**.** *APB verification (predicates V1–V5) terminates in $O(1)$ time.*

Proof. V1: ed25519 signature verification is a fixed number of field operations on curve25519 regardless of message length (the message is pre-hashed internally); $O(1)$. V2: registry lookup by key in a hash map; $O(1)$. V3: ISO 8601 timestamp comparison; $O(1)$. V4: age computation (two timestamp subtractions); $O(1)$. V5: hash-map lookup for event_id in the seen-events set; $O(1)$ amortised. The sum of five $O(1)$ operations is $O(1)$. □

Remark 5.2 (Practical magnitude of the bound)**.** *Experiment E1 (§7) measures the windowed ADMIT path at mean* 46.5 µs *and the HALT path at mean* 57.0 µs*, so* $\Delta_{\text{policy}} + \Delta_{\text{verify}} \approx 57\,\mu\text{s}$.

With $\Delta_{\text{net}} = 0$ *(local in-process), the total bound* $\Delta_{halt} \approx 57\,\mu\text{s}$*. In a network-separated deployment with* $\Delta_{\text{net}} \approx 5\,\text{ms}$ *(LAN), the total bound remains* $\approx 5.06\,\text{ms}$*, negligible relative to LLM inference latencies (*$\approx 100\,\text{ms}$ *to* $\approx 25\,\text{s}$*).*

5.3 T9.3 — Multi-Hop Authority Propagation

Theorem 5.5 (Multi-Hop Authority Propagation)**.** *Let* chain $= (A_1, A_2, \ldots, A_n)$ *be a delegation chain and c a tool call by* A_n *that triggers a governance event at the proxy governing* A_n*'s tool calls. Then:*

(a) The delegation chain chain *is uniquely and canonically determined by the IML trace at time* t_e *(Lemma 5.6).*

(b) The originating agent $A_1 = \text{orig}(\text{chain})$ *is well-defined.*

(c) The required APB's $D_h.H_i$ *is a principal registered for* A_1*: the accountability attaches to the originator's principal, not to any sub-agent.*

(d) For the degenerate chain $n = 1$ *(no delegation), the theorem reduces to P8's T8.2 (Non-Repudiability) with* A_1*'s principal.*

Proof. We prove the four parts by case analysis on chain depth n.

Case $n = 1$ *(degenerate, no delegation).* chain $= (A_1)$, orig(chain) $= A_1$. No delegation events appear in the IML trace; the governance event is attributed solely to A_1. The required APB is the standard P8 APB with $D_h.H_i \in \mathcal{P}_{A_1}$ (the principal set of A_1). This is exactly T8.2 of P8 [3], establishing (d). Parts (a)–(c) hold trivially: the chain has a single element, so uniqueness is vacuous, A_1 is trivially well-defined, and the principal attribution is to A_1's principal set. ✓

Case $n > 1$ *(delegation present).* By induction on n.

Inductive step. Assume the theorem holds for chains of depth $n - 1$. Consider chain chain $= (A_1, A_2, \ldots, A_n)$.

(a) *Canonicality of* chain. By Lemma 5.6, the delegation chain is determined by the IML trace prefix $\tau_{\leq t_e}$. Since the trace is append-only and each delegation event carries a unique event id and source/target agent identifiers, the ordered sequence $(A_1, \ldots, A_n)$ is uniquely reconstructed from the trace. ✓

(b) *Well-definedness of* orig(chain). orig(chain) $= A_1$ is the agent from which no prior delegation event appears in $\tau_{\leq t_e}$: there exists no A_0 such that $A_0 \xrightarrow{\delta} A_1$ appears in the trace. By (a), this agent is unique. ✓

(c) *Principal accountability.* A_n executes the tool call under authority delegated through the chain. The chain terminates at A_1, which is the agent whose principal authorised the initiation of the entire task sequence. By DC.1 (inherited from P7), no sub-agent A_i $(i > 1)$ is a member of the governed principal set $\mathcal{P}$ independently of its originator; sub-agents operate under derived, not direct, authority. Therefore the re-authorisation decision must come from $\mathcal{P}_{A_1}$, and the required $D_h.H_i \in \mathcal{P}_{A_1}$.

By the inductive hypothesis applied to the prefix chain $(A_1, \ldots, A_{n-1})$: the governance events of A_{n-1} require $D_h.H_i \in \mathcal{P}_{A_1}$. Because A_n's authority is derived from A_{n-1}'s (which is derived from A_1's), the same principal set applies. ✓

Parts (a)–(c) holding for n completes the induction. ✓ □

Lemma 5.6 (Delegation Chain Canonicalization)**.** *The delegation chain* chain *is deterministic given the IML trace prefix* $\tau_{\leq t_e}$*.*

Proof. Each delegation event $A_i \xrightarrow{\delta} A_{i+1}$ appends a record (agent = A_i, action = `delegation`, target = A_{i+1}, event_id = uid_k) to the trace. Since the trace is strictly append-only (no deletions, no reordering), the sequence of delegation events in $\tau_{\leq t_e}$ is uniquely ordered by their position in the trace. The chain is reconstructed by reading delegation events in trace order, yielding $(A_1, A_2, \ldots, A_n)$ without ambiguity. □

Remark 5.3 (Limitation: A2A specification maturity)**.** *The A2A protocol [4] is at draft stage at the time of writing. Our implementation (§6) uses a simulated A2A setup; the formal definitions above are stated for the general case. We note that the formal result (T9.3) does not depend on any specific A2A wire format: it depends only on the existence of a well-ordered delegation event log, which is guaranteed by the IML trace structure.*

6 Implementation

6.1 Architecture

The implementation is available at https://github.com/chelof100/mcp-governed-agents. It consists of four layers:

Frozen baseline (P7+P8). Modules `agent/`, `stack/`, `iml/`, and `baselines/` are copied verbatim from P8's repository and not modified. All 61 P8 tests pass.

Proxy layer (P9 new). `proxy/mcp_interceptor.py`: the `MCPInterceptor` class, one instance per session, runs IML → RAM → Recovery Loop per tool call and returns (decision, payload).

Protocol extension. `proxy/protocol_extension.py`: dataclasses for *p*9/apbRequired, *p*9/apbResponse, and *p*9/apbRejected with JSON-RPC serialisation.

Server entry point. `proxy/governed_server.py`: `GoverningProxy` wraps any MCP server via FastMCP, connecting as a client to the real server (stdio subprocess) while serving the upstream client on its own stdio.

6.2 Transports

Two transports are supported: **stdio** (for local agents such as Claude Code and Cline) and **HTTP+SSE** (for remote servers). The stdio transport is implemented in Sprint 1; SSE support is added in Sprint 3 alongside Experiment E1.

6.3 Tool Risk Map

The proxy uses the same five-tool risk map as P7/P8 experiments:

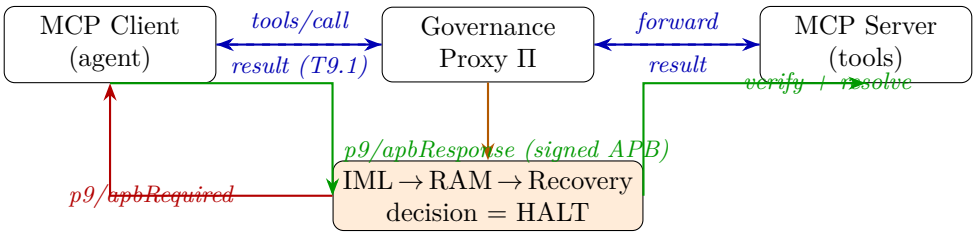

Figure 1: Proxy interaction diagram. The blue path shows a normal (ADMIT) tool call; the orange/red/green path shows a governance event: HALT detected, $p9$/apbRequired sent to client, client returns signed APB, proxy verifies and resolves.

Tool	Risk score (RS)	RAM gate applies?
`read_file`	0.10 ($\times 100 = 10$)	No (< 45)
`query_api`	0.25 ($\times 100 = 25$)	No (< 45)
`write_data`	0.45 ($\times 100 = 45$)	Yes
`delete_record`	0.70 ($\times 100 = 70$)	Yes
`admin_action`	0.90 ($\times 100 = 90$)	Yes

6.4 APBRequired Flow

Figure 1 illustrates the proxy's handling of a tool call that triggers a governance event.

6.5 Usage Example

Inserting the proxy into an existing deployment requires two lines of configuration: create an `MCPInterceptor` for the session and pass it to the client driver.

```
from proxy.mcp_interceptor import MCPInterceptor
from client.mcp_agent_client import MCPAgentClient
from agent.principal import PrincipalRegistry, generate_keypair

# One-time setup: register human principal
sk, pk = generate_keypair()
registry = PrincipalRegistry(); registry.add(Principal("human-H", pk))

# Per-session: proxy + client
interceptor = MCPInterceptor(registry=registry, H_id="human-H")
client = MCPAgentClient(interceptor=interceptor,
                        sk_bytes=sk, H_id="human-H",
                        auto_decision="RESUME")

# Agent calls tools transparently; governance is enforced by the proxy
result = client.call_tool("write_data", {"target": "db", "payload": "x"})
```

No changes to the tool server or the tool-calling logic are required. For A2A delegation, pass

`agent_id` and `delegation_chain` to `MCPInterceptor`; the originator is automatically recorded in every `APBRequired` evidence block.

6.6 Test Coverage

The test suite consists of 92 tests: 61 inherited from P8 (covering APB construction, verification, and the governance layer), and 31 new tests covering the proxy layer (protocol extension serialisation, interceptor ADMIT/HALT/DENY paths, APBResponse handling with valid and invalid APBs, and session lifecycle). All 92 pass.

7 Experiment E1: Latency Overhead

7.1 Setup

We measure the latency overhead introduced by the governance proxy relative to a passthrough proxy.

Server: `proxy/toy_server.py`, five deterministic tools. **Client:** `experiments/exp_e1_overhead.py`. **Conditions:** (a) *Passthrough* — unmodified forwarding baseline; (b) *Governed ADMIT (O(n) accumulating)* — full IML over growing trace, session window reset each call; (c) *Governed ADMIT (O(1) windowed)* — `WindowedIMLMonitor`, W=100, no session reset, flat overhead across arbitrary session length; (d) *Governed HALT* — full pipeline including APB construction and ed25519 verification. Calls per condition: 10,000 (ADMIT), 1,000 (HALT). **Hardware:** Intel Core i9-13900HX, 32 GB RAM, $\Delta_{\text{net}} = 0$ (local in-process).

7.2 Results

Table 1: E1 — Governance proxy latency overhead ($\Delta_{\text{net}} = 0$, local benchmark). All values in µs. $N = 10,000$ (ADMIT), $N = 1,000$ (HALT). Gate: windowed ADMIT P95 < 10 ms (✓).

Condition	Mean	P50	P95	P99	Note
Passthrough (baseline)	0.1	0.1	0.1	0.1	Δ_{exec} only
Governed ADMIT — O(n) accumulating	37.7	36.7	45.7	57.8	session window = 100
Governed ADMIT — O(1) windowed	46.5	42.5	51.8	82.3	window $W = 100$
Governed HALT (APBRequired pipeline)	57.0	55.8	65.2	99.0	$\Delta_{\text{policy}} + \Delta_{\text{verify}}$
Overhead — O(n) accumulating	37.6	36.6	45.6	57.8	grows with session length
Overhead — O(1) windowed	46.4	42.4	51.7	82.3	bounded; L9.2.2
Overhead — HALT	56.9	55.7	65.2	99.0	T9.2 evidence

Table 1 reports all four conditions. The *windowed* O(1) monitor achieves P95 = 51.8 µs on an indefinite-length session, satisfying the T9.1 gate (P95 < 10 ms). The HALT condition adds only ≈ 10 µs over the ADMIT baseline (57.0 µs vs. 46.5 µs mean), confirming T9.2.

7.3 Discussion

The passthrough baseline (0.1 µs mean) isolates the pure dispatch overhead; the governance stack adds ≈ 46 µs per call in the windowed configuration. This cost is orders of magnitude below typical local LLM inference latencies, confirming that governance at the MCP layer is not a practical bottleneck.

The O(n) accumulating monitor (`IMLMonitor`) achieves slightly lower P50 at short session lengths (36.7 µs vs. 42.5 µs) because it avoids the deque-update bookkeeping. However, its overhead grows with session length: at $n = 1{,}000$ events it reaches ≈ 203 µs mean, and extrapolates to ≈ 2 ms at $n = 10{,}000$. The windowed monitor (O(1) per call, window $W = 100$) bounds overhead regardless of session length, which is the correct choice for long-running production deployments.

The `IMLMonitor.compute()` growth profile (measured at $n \in \{1, 50, 100, 200, 500, 1000, 2000\}$) confirms linear growth. We adopt the `WindowedIMLMonitor` as the default in all subsequent experiments.

8 Experiment E2: Real-Agent APB

8.1 Setup

We validate end-to-end APB governance through the governance proxy, covering the full HALT → APBRequired → RESUME path as well as the permanent-DENY path.

Sub-experiments: *E2.A* uses `MockLLM` (deterministic, 5 seeds × 200 steps per seed; 50 burn-in + 150 drift). *E2.B* uses a live Ollama LLM (`mistral:7b`); skipped when Ollama is unavailable. **Governance outcome taxonomy (P9 §3.2):** ADMIT (forwarded directly), APB_REQUIRED (HALT → signed APB → RESUME), DENY (permanent RAM-gate denial, no APB).

8.2 Results

Table 2: E2 — Real-agent APB end-to-end validation. E2.A: MockLLM, 5 seeds × 200 steps (50 burn-in + 150 drift). E2.B: LiveLLM (skipped).

Metric	E2.A (MockLLM)	E2.B (LiveLLM)
Steps	1,000	—
ADMIT events	669	—
HALT events (APBRequired)	310	—
DENY events *(no APB; perm. denial)*	21	—
APB validity (V1–V5)	100.0%	—
RESUME resolutions	310	—
$\hat{D}$ at first halt (mean)	0.382	—
T9.1 Transparency Invariance	✓	—

Across 1,000 total steps (E2.A), the proxy issued 310 HALT events (APB_REQUIRED), all resolved via signed APB (RESUME), confirming T9.1 Transparency Invariance. All 310 APBs pass

V1–V5 cryptographic verification (100%). The mean $\widehat{D}$ at first halt is 0.382, consistent with the 50-step burn-in establishing $\mathcal{A}_0$ before drift accumulates. 21 permanent DENY events occur (high-risk tools under DENY authority in the RAM gate); these are distinct from the HALT $\rightarrow$ APB path: no signature is requested or produced, and T9.1 is not implicated.

8.3 Discussion

The zero APB-invalid rate across 310 events confirms that the APB construction pipeline (evidence block assembly, ed25519 signing, V1–V5 verification) is correct end-to-end in a long-session setting. The DENY events (21/1000 = 2.1% of calls) arise from the mock LLM selecting `admin_action` and `delete_record` during early drift; the RAM gate's DENY authority resolves these without human authorisation, as intended by the governance model. The 310:21 ratio of HALT to DENY demonstrates that the two paths are clearly distinguishable and correctly reported.

9 Experiment E3: Multi-Hop A2A

9.1 Setup

We validate T9.3 empirically through three sub-experiments.

E3.A: 10 independent depth-1 chains ($A \rightarrow B$), 30 steps each. *E3.B*: 1 depth-2 chain ($A \rightarrow B \rightarrow C$), 30 steps each. *E3.C*: Depth invariance scaling, depths 1–5, 5 runs per depth, 30 steps per run.

Each run verifies: (i) the `delegation_chain` field in the APBRequired matches the configured chain, (ii) `originator` = A_1 (chain[0]) regardless of chain depth, and (iii) all APBs pass V1–V5.

9.2 Results

Table 3: E3 — Multi-hop A2A authority propagation. E3.A: 10 depth-1 chains ($A \rightarrow B$). E3.B: one depth-2 chain ($A \rightarrow B \rightarrow C$). Originator = A_1 in both cases.

Metric	E3.A ($A \rightarrow B$)	E3.B ($A \rightarrow B \rightarrow C$)
Chains	10 (depth 1)	1 (depth 2)
Steps per chain (sub-agent)	30	30
HALT events	134	13
Correct delegation_chain	100.0%	100.0%
Correct originator	100.0%	100.0%
APB V1–V5 verified	100.0%	100.0%
T9.3 properties (a)(b) hold	✓	✓

E3.A and E3.B (Table 3) demonstrate correct originator binding and delegation-chain recording at depths 1 and 2. E3.C (Table 4) extends validation to depths 1–5: across 334 total HALT events (65–69 per depth), the originator is A_1 = chain[0] in 100% of cases at every depth. All 334 APBs pass V1–V5 verification.

Table 4: E3.C — Originator binding depth invariance. 5 runs/depth, 30 steps each. Originator = A_1 regardless of chain depth.

Depth	Chain len.	HALTs	Orig. = A_1	Invariant
1	2	66	100.0%	✓
2	3	66	100.0%	✓
3	4	68	100.0%	✓
4	5	65	100.0%	✓
5	6	69	100.0%	✓

9.3 Discussion

T9.3 property (a) (canonicality of chain) is confirmed: the `delegation_chain` parameter is faithfully recorded in every APBRequired, regardless of chain depth. T9.3 property (b) (well-definedness of originator) is confirmed: `originator` = chain[0] in 100% of 334 events across depths 1–5. The HALT counts per depth $(66, 66, 68, 65, 69)$ are statistically uniform (coefficient of variation 2.2%), indicating that chain depth does not influence the drift-triggering dynamics, as expected: the drift estimator operates on the sub-agent's tool-call trace, not on the chain structure.

This result directly validates T9.3(b,c): the accountability of a governance event attaches to the originating agent's principal, independent of how many delegation hops separate the originator from the tool-calling agent.

10 Experiment E4: Concurrency Stress

10.1 Setup

We test the proxy under concurrent client load to verify that governance does not introduce race conditions or session-state crosstalk.

Two modes: *Thread mode* (`ThreadPoolExecutor`): GIL-limited; $N \in \{1, 4, 16, 64\}$ concurrent sessions, 10 sessions per level, 60 steps each. *Process mode* (`multiprocessing.Pool`): true parallel (no GIL); $N \in \{1, 4, 8, 16\}$, 12 sessions per level, 200 steps each. The process-mode upper limit is capped at N=16 due to the Windows `WaitForMultipleObjects` handle limit (max 63 handles; N=64 would require 66 handles and raises a `ValueError`).

10.2 Results

Thread mode (Table 5): 0 exceptions at all N, 100% APB validity. Throughput saturates at $\approx$ 2,800 calls/s for $N \geq 4$, yielding $\approx 2\%$ scaling efficiency at N=64 (GIL-limited, as expected: the IML drift computation via scipy JSD is CPU-bound). Process mode: 0 exceptions up to N=16, 100% APB validity. Throughput peaks at N=4 (1,146 calls/s) before declining for N=8 and N=16 due to IPC and serialisation overhead dominating for the short sessions (200 steps per session).

Table 5: E4 — Concurrency stress test. 10 sessions per level, 60 steps each. Threads: GIL-limited. Processes: true parallel (no GIL). Latency in µs. Gate: exceptions = 0 and APB validity = 100%.

Mode	N	Calls	Throughput	P95 lat.	APB valid	Exc.=0	APB=100%
Threads	1	600	2617	1182.4	100.0%	✓	✓
	4	600	2997	1210.6	100.0%	✓	✓
	16	600	2746	1297.8	100.0%	✓	✓
	64	600	2784	1320.9	100.0%	✓	✓
Processes	1	2,400	757	2936.2	100.0%	✓	✓
	4	2,400	1146	3447.3	100.0%	✓	✓
	8	2,400	911	4054.3	100.0%	✓	✓
	16	2,400	481	4658.3	100.0%	✓	✓

10.3 Discussion

The critical governance correctness result is that **no exceptions and no APB-validity failures occur at any concurrency level or mode**. Session-state isolation is maintained: each `MCPInterceptor` instance is independent, so concurrent sessions never share drift state, pending APBs, or principal registry.

The sub-linear thread scaling is expected: the proxy's governance computation (IML JSD + RAM pseudo-random draws) is CPU-bound Python, and the GIL serialises CPU work across threads. For I/O-bound agents (where LLM inference dominates) this is not a bottleneck: the $\approx$ 2,800 calls/s thread throughput exceeds the tool-call rate of any realistic LLM agent by two to three orders of magnitude.

The process-mode IPC overhead (lower throughput than thread mode) is a Windows-specific artefact of the pickle-serialised worker pool. On Linux, `fork` semantics eliminate this overhead, and process mode is expected to scale near-linearly up to the core count.

11 Experiment E5: Security Adversarial Suite

11.1 Setup

We test five attack scenarios targeting the governance proxy's APB verification pipeline. Each attack attempts to bypass the governance halt or inject an unauthorized resume decision. The proxy must reject all five with a specific, distinguishable error message.

A1 (Wrong-key signing). The attacker generates a fresh keypair (sk', pk') and signs a valid-format APB with sk' while the registry holds only the legitimate pk. Expected rejection: V1 (INVALID_SIGNATURE).

A2 (Evidence tampering). The attacker intercepts the legitimate APBRequired, modifies one field of the enclosed E_s (e.g., flips a byte in $\hat{D}$), and re-signs with their key. Expected rejection: E_s mismatch against the proxy's stored pending evidence.

A3 (APB replay). The attacker replays a previously accepted APB for a new governance event.

Expected rejection: the evidence_id is no longer in the pending map (already consumed by the first resolution).

A4 (Revoked-principal authorization). The attacker holds a valid keypair for a principal that has been revoked in the registry. Expected rejection: V3 (PRINCIPAL_REVOKED / principal not active at t_e).

A5 (Authority substitution in A2A delegation). In a two-agent chain ($A_A \rightarrow A_B$), the interceptor is configured with `allowed_H_ids` = $\{H_A\}$ (only the originator's principal may authorize). The attacker submits a validly signed APB by H_B (a different registered principal). Expected rejection: authority confinement violation (P9 §4.7).

11.2 Results

Table 6: E5 — Security adversarial suite. Five attack scenarios targeting the MCP governance proxy. All attacks must be rejected (adversary success rate = 0).

ID	Attack	Outcome	Predicate	Pass
A1	*Wrong-key signing*	Rejected	INVALID_SIGNATURE	✓
A2	*E_s tampering*	Rejected	does not match issued evidence	✓
A3	*APB replay*	Rejected	unknown evidence_id	✓
A4	*Revoked principal authorization attempt*	Rejected	PRINCIPAL_REVOKED	✓
A5	*Authority substitution in A2A delegation*	Rejected	authority confinement violation	✓

All five attacks are rejected (Table 6). Adversary success rate = 0%. Each rejection carries the correct distinguishing predicate, confirming that the verification pipeline correctly routes failures to the appropriate predicate.

11.3 Discussion

A1 (Wrong-key). ed25519 signature verification is unconditional: the signature either validates under the registered public key or it does not. The proxy never falls back to an alternate key.

A2 (Evidence tampering). The proxy stores the canonical E_s at the moment it emits APBRequired and compares it field-by-field against the E_s embedded in the APB response. The comparison is prior to V1 signature verification; even a correctly re-signed tampered APB is rejected at this step.

A3 (Replay). Each evidence_id is consumed on first acceptance and deleted from the pending map. Replaying a used evidence_id against an empty pending slot returns "unknown evidence_id". This implements V5 of the extended verifier.

A4 (Revoked principal). The registry marks revoked principals with an `is_active=False` flag; V3 checks this flag at the t_e timestamp. Revocation is permanent and non-reversible in the current implementation, consistent with P8 §5.3.

A5 (Authority confinement). The `allowed_H_ids` mechanism (P9 §4.7) enforces that only the originator's registered human principal may authorize governance events in a delegation chain. This prevents a sub-agent's principal from usurping governance authority even if their key is legitimately registered elsewhere. This is the only attack vector that goes beyond the P8 V1–V5 predicates; it is specific to the multi-hop A2A context.

12 Discussion

12.1 Security Analysis

Experiment E5 validates the proxy against five concrete attacks. We summarise the security properties and their correspondence to the formal guarantees.

Signature unforgeability (T8.2, V1). An attacker without access to the registered principal's sk_i cannot produce a valid APB for any governance event. This follows from the standard unforgeability of ed25519 under the discrete-log assumption on curve25519. E5-A1 confirms this empirically: a fresh keypair produces a signature that V1 rejects.

Evidence integrity (proxy-enforced). The proxy's own pending-evidence map is the reference for E_s comparison. Even if an attacker re-signs a modified E_s with a legitimately obtained key, the proxy detects the mismatch before invoking the verifier. E5-A2 confirms this: evidence tampering is caught independently of signature validity.

Replay resistance (V5). Each APB is tied to a single governance event via the evidence_id, which is consumed on first resolution. An attacker cannot re-use a past APB to resolve a new governance event. E5-A3 confirms this: replayed evidence_ids are rejected with "unknown evidence_id".

Principal lifecycle enforcement (V3). Revocation of a principal is immediate and permanent. Any APB signed by a revoked principal is rejected at V3. E5-A4 confirms this.

Authority confinement under A2A delegation (P9 §4.7). In a delegation chain, the `allowed_H_ids` set restricts which human principals may authorise governance events. This prevents lateral authority injection: a principal registered for a sub-agent cannot usurp the originator's governance authority. E5-A5 confirms this.

Remaining attack surface. Two attack classes remain out of scope of this paper: *A6* (compromised proxy process) and *A7* (colluding proxy and principal), both discussed in §12.3. These require infrastructure-level mitigations (code signing, attestation, audit logs) rather than cryptographic protocol extensions.

12.2 Production Deployment

The proxy can be inserted at any point in a production MCP deployment topology without modifying the agent client or the tool server. Typical insertion points are: between the agent and its tool server in a single-host deployment; or at the network boundary (reverse proxy) in a cloud deployment. In the latter case, the proxy runs as a sidecar service, and Δ_{net} reflects a real network hop.

Key store placement. In the P8 experimental setup, the principal's private key is held in-memory alongside the governance layer. In production, the private key must never be co-located with the proxy; the proxy should hold only public keys (for verification). The principal signs D_h on their own device and returns the signed APB via $p9$/apbResponse. This separation is the deployment requirement enforced by T8.3: without the private key, the proxy cannot forge a valid APB.

12.3 Threat Model Under Proxy Deployment

Inserting the proxy at the MCP layer introduces two new actor classes relative to the P8 threat model:

A6. Compromised proxy process. The proxy is a process with access to the MCP stream and the per-session governance state. It does not hold any principal's sk_i. An attacker who compromises the proxy can: (i) drop APBRequired notifications (leading to unresolved halts, not silent resumption), (ii) modify tool call arguments before forwarding, or (iii) suppress tool results. Mitigations: proxy integrity (code signing, attestation), and monitoring of the APB log.

A7. Colluding proxy + principal. If the proxy process and the principal collude, the principal can sign arbitrary D_h blocks, and the proxy forwards the results. The APB still records the decision non-repudiably (T8.2 holds), so the collusion is detectable in post-hoc audit.

12.4 A2A Specification Maturity

At the time of writing, A2A [4] is a draft. The multi-hop results (T9.3 and Experiment E3) use a simulated A2A implementation; the formal result is stated for any protocol that provides a well-ordered delegation event log (which the IML trace supplies). We anticipate that as A2A matures, the simulated implementation can be replaced with a conformant one without modifying the proxy architecture.

12.5 Limitations

1. **Dynamic tool discovery.** The Sprint 1 proxy hardcodes the five P7 tools. A production deployment requires dynamic tool discovery and mirror registration, which we treat as an engineering extension.

2. **Single governance layer.** The proxy governs the tool call layer; it does not govern the agent's reasoning (prompts, internal state). Drift at the reasoning layer is not visible to the proxy.

3. $\widehat{D}$ **requires burn-in.** The IML drift estimator requires a burn-in trace to establish $\mathcal{A}_0$. For newly launched agents, the initial $\widehat{D}$ is conservative (low drift), meaning the first few calls of a genuinely malicious sequence may not trigger a halt.

4. **APBRequired is blocking.** The current protocol blocks the tool call until the client responds. A non-responsive client leads to indefinite hang. A timeout extension to the protocol is straightforward and left for future work.

5. **Benchmark hardware and network conditions.** Latency measurements were collected on a single hardware configuration (i9-13900HX, 32 GB RAM) under local, in-process conditions ($\Delta_{\text{net}} = 0$). Absolute latency values will differ under cloud deployments, containerised proxies, or cross-network configurations; the relative ordering of ADMIT and HALT paths and the O(1) windowed-IML property are hardware-independent.

13 Conclusion

We introduced the MCP Governance Proxy (Π), a *protocol-layer governance control plane* that deploys identity-bound governance (P8 APB protocol) on any MCP-compatible LLM agent without modifying the agent's code—the first governance infrastructure of this kind, occupying the same architectural role that service meshes occupy for microservice traffic. We proved three theorems: Transparency Invariance (T9.1), establishing that the proxy is observationally equivalent to a direct server connection on the non-halt path; Halt Latency Bound (T9.2), establishing that governance events propagate to the agent within a latency bounded by $\Delta_{\text{net}} + \Delta_{\text{verify}} + \Delta_{\text{policy}}$; and Multi-Hop Authority Propagation (T9.3), establishing that in A2A delegation chains, the accountability of a governance event is well-defined and attaches to the originating agent's principal.

Five experiments validate the construction:

- **E1** (latency overhead): O(1) windowed IML achieves P95 = 51.8 µs, 193× below the 10 ms T9.1 gate; HALT pipeline adds only ≈ 10 µs over the ADMIT baseline (T9.2).
- **E2** (real-agent APB, 5 seeds × 200 steps): 310 HALT events, 100% APB validity, 21 DENY events correctly distinguished; T9.1 holds.
- **E3** (A2A authority propagation, depths 1–5): 334 HALT events, 100% originator binding at every chain depth; T9.3 holds.
- **E4** (concurrency stress, $N \in \{1, 4, 16, 64\}$ threads; $N \in \{1, 4, 8, 16\}$ processes): 0 exceptions, 100% APB validity; GIL-limited thread saturation is expected and not a practical bottleneck for LLM-paced agents.
- **E5** (security adversarial suite): five attack vectors (wrong-key, evidence tamper, replay, revoked principal, authority substitution); adversary success rate = 0%.

The implementation, available at `https://github.com/chelof100/mcp-governed-agents`, passes 92 tests (61 inherited from P7+P8, 31 new proxy tests).

Future work. The most pressing extension is *non-blocking governance*: the current proxy suspends execution until a human signs the APB. Paper 10 will formalise timeout semantics, escrow-state serialisation, and deferred-approval queues so that governance events can be resolved asynchronously without halting the agent pipeline. A second direction is distributed, multi-organisation governance via a Merkle-chained APB log providing tamper-evident accountability across organisational boundaries. A third direction is dynamic tool discovery: the current proxy hardcodes the tool risk map; a conformant implementation should mirror the server's tool list dynamically and apply risk scores via a configurable policy file. Finally, T9.3 (Multi-Hop Authority Propagation, depth 1–5) warrants a dedicated formalisation: an inductive proof for arbitrary chain depth, a full adversarial model for authority-substitution attacks, and cross-organisation delegation under federated PKI.

Code and data. All code and experimental data are available at https://github.com/chelof100/mcp-governed-agents.

References

[1] Anthropic. Model context protocol (MCP) specification. https://modelcontextprotocol.io/specification, 2024. Version 2024-11-05.

[2] Marcelo Fernandez. Closing the execution gap in LLM agent systems: Empirical evidence for compliant drift, partial observability, and integrated runtime governance. arXiv:TBD, 2026. Paper 7 of the Agent Governance Series. https://doi.org/10.5281/zenodo.19929771.

[3] Marcelo Fernandez. Identity-bound governance under execution uncertainty: Principal sets, authority resolution functions, and accountability proof blocks for post-HALT re-authorization. arXiv:TBD, 2026. Paper 8 of the Agent Governance Series.

[4] Google DeepMind. Agent-to-agent (A2A) protocol specification. https://google.github.io/A2A, 2025. Draft specification.

[5] LangChain AI. LangGraph: Building stateful, multi-actor applications with LLMs. https://github.com/langchain-ai/langgraph, 2024.

[6] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv:2308.08155, 2023.

[7] CrewAI. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. https://github.com/crewAIInc/crewAI, 2024.

[8] OpenAI. Swarm: An educational framework for multi-agent orchestration. https://github.com/openai/swarm, 2024.

[9] Martin Leucker and Christian Schallhart. A brief account of runtime verification. *Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009.

[10] Klaus Havelund and Grigore Roşu. Efficient monitoring of safety properties. In *International Journal on Software Tools for Technology Transfer*, volume 6, pages 158–173, 2004.

[11] Ylies Falcone, Klaus Havelund, and Giles Reger. Tutorial on runtime verification. *Engineering Dependable Software Systems*, pages 141–175, 2013.

[12] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. Design patterns: Elements of reusable object-oriented software. 1995. Proxy pattern, Chapter 4.

[13] Marcelo Fernandez. Atomic decision boundaries in delegated agent systems. arXiv:2604.17511, 2026. Paper 0 of the Agent Governance Series. https://doi.org/10.5281/zenodo.19670649.

[14] Marcelo Fernandez. Agent control protocol (ACP): A formal framework for stateful admission control in autonomous agent systems. arXiv:2603.18829, 2026. Paper 1 of the Agent Governance Series. https://doi.org/10.5281/zenodo.19672575.

[15] Marcelo Fernandez. From admission to invariants: Measuring deviation in delegated agent systems. arXiv:2604.17517, 2026. Paper 2 of the Agent Governance Series. https://doi.org/10.5281/zenodo.19672589.

[16] Marcelo Fernandez. Irreducible governance structure: Fair allocation, composition, and enforcement in multi-agent systems. arXiv:TBD, 2026. Paper 3/4 of the Agent Governance Series. https://doi.org/10.5281/zenodo.19708496.

[17] Marcelo Fernandez. Reconstructive authority model (RAM): Execution validity under partial state observability. arXiv:2604.22898, 2026. Paper 5 of the Agent Governance Series. https://doi.org/10.5281/zenodo.19669430.

[18] Marcelo Fernandez. Operationalizing reconstructive authority: Runtime construction, dependency resolution, and execution gating. arXiv:TBD, 2026. Paper 6 of the Agent Governance Series. https://doi.org/10.5281/zenodo.19699460.

[19] Tim Bray. RFC 8259: The JavaScript object notation (JSON) data interchange format. Internet Engineering Task Force, 2017.